

NAGP 2026 - Assignment Documentation

Kubernetes, DevOps & FinOps Workshop

Technology Band III Batch

Table of Contents

1. [Requirement Understanding](#)
 2. [Assumptions](#)
 3. [Solution Overview](#)
 4. [Justification for Resources Utilized](#)
 5. [Deployment Instructions](#)
 6. [Demonstration Guide](#)
 7. [FinOps Analysis](#)
-

1. Requirement Understanding

1.1 Problem Statement Summary

The assignment requires designing, containerizing, and deploying a **multi-tier architecture on Kubernetes** with:

- **Service API Tier:** A microservice that exposes API endpoints and fetches data from the database tier
- **Database Tier:** A database with persistent storage containing sample data

1.2 Service API Tier Requirements

Requirement	Description	Compliance
API Endpoint	Expose an API endpoint accessible externally	Implemented via Ingress
Data Retrieval	Fetch data from database tier	/api/employees endpoint
Technology	Any language/framework	Python Flask
Best Practices	Connection pooling, config separation	psycopg2 pool, ConfigMap
Rolling Updates	Support for zero-downtime deployments	RollingUpdate strategy
External Access	Must be accessible from outside cluster	NGINX Ingress
Self-Healing	Automatic recovery from failures	Liveness/Readiness probes
HPA	Horizontal Pod Autoscaler	CPU/Memory based scaling
Pods	4 replicas	Deployment with 4 replicas

Requirement	Description	Compliance
ConfigMap Usage	Yes	Database configuration
Secrets Usage	Yes	Database password

1.3 Database Tier Requirements

Requirement	Description	Compliance
Sample Data	One table with 5-10 records	8 employee records
Persistence	Data should survive pod restarts	PersistentVolumeClaim
Internal Only	Not accessible from outside cluster	ClusterIP Service
Self-Healing	Automatic recovery after pod deletion	Deployment controller
Pods	1 replica	Deployment with 1 replica
Secrets Usage	Yes	Database credentials

1.4 Additional Requirements

Requirement	Implementation
No Pod IPs for communication	Kubernetes Service DNS names used
Password not visible in YAML	Base64 encoded in Kubernetes Secrets
External access via Ingress	NGINX Ingress Controller on AKS
Resource requests/limits	Defined for all containers
Cost optimization	Three+ optimization opportunities identified

2. Assumptions

2.1 Infrastructure Assumptions

1. **Cloud Platform:** Microsoft Azure with Azure Kubernetes Service (AKS)
2. **Cluster Configuration:**
 - Standard AKS cluster
 - Minimum 2 nodes for high availability
 - VM size: chosen per budget (e.g., 2 vCPU / 4GB RAM class)
3. **Storage:** AKS default StorageClass (Azure Disk CSI), typically `managed-csi`

2.2 Application Assumptions

1. **Tech Stack:** Python 3.11 with Flask framework (chosen for simplicity and wide adoption)
2. **Database:** PostgreSQL 15 (reliable, production-ready, excellent Python support)
3. **Container Registry:** Docker Hub for storing container images

4. **API Format:** RESTful JSON APIs

2.3 Security Assumptions

1. For assignment purposes, database credentials are stored in Kubernetes Secrets with base64 encoding
2. In production, would use:
 - Azure Key Vault + Secrets Store CSI Driver
 - External Secrets Operator
 - HashiCorp Vault

2.4 Network Assumptions

1. AKS default network configuration
2. No custom VPC or firewall rules required beyond defaults
3. Ingress controller: NGINX Ingress Controller

2.5 Scale Assumptions

1. Initial load: Low to moderate (4 API pods sufficient)
2. HPA configured to scale up to 10 pods for traffic spikes
3. Single database replica sufficient for this use case

3. Solution Overview

3.1 Architecture Components

3.1.1 Service API Tier

Technology: Python 3.11 + Flask + Gunicorn

Key Features:

- **Flask Application:** Lightweight web framework for REST API
- **Gunicorn WSGI Server:** Production-grade application server
- **Connection Pooling:** psycopg2 ThreadedConnectionPool for efficient database connections
- **Configuration Separation:** All database settings via environment variables (ConfigMap/Secrets)

Deployment Strategy:

- Kubernetes Deployment with 4 replicas
- RollingUpdate strategy (maxSurge: 1, maxUnavailable: 1)
- Health checks via `/health` endpoint
- Readiness checks via `/ready` endpoint (includes DB connectivity)

3.1.2 Database Tier

Technology: PostgreSQL 15 Alpine

Key Features:

- **Deployment:** Single replica, simple rollout for assignment/demo
- **Persistent Storage:** 1GB PVC with AKS `managed-csi` StorageClass
- **Internal Access Only:** ClusterIP Service (no external exposure)

Data Schema:

```
CREATE TABLE employees (  
  id SERIAL PRIMARY KEY,  
  name VARCHAR(100) NOT NULL,  
  department VARCHAR(100) NOT NULL,  
  position VARCHAR(100) NOT NULL,  
  email VARCHAR(150) NOT NULL,  
  salary DECIMAL(10, 2) NOT NULL,  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

Sample Data: 8 employee records automatically seeded on first startup

3.1.3 Configuration Management

ConfigMap (`db-config`):

- DB_HOST: postgres-service (Kubernetes Service DNS)
- DB_PORT: 5432
- DB_NAME: nagpdb
- DB_USER: nagpuser
- DB_POOL_MIN_CONN: 1
- DB_POOL_MAX_CONN: 10

Secret (`db-secret`):

- DB_PASSWORD: Base64 encoded
- POSTGRES_USER: Base64 encoded
- POSTGRES_PASSWORD: Base64 encoded
- POSTGRES_DB: Base64 encoded

3.1.4 Networking

Service API Service (ClusterIP):

- Internal access for Ingress controller
- Port 80 → Target Port 5000

PostgreSQL Service (ClusterIP):

- Internal access only
- Port 5432 → Target Port 5432

Ingress (NGINX):

- External HTTP access
- Path: / → service-api:80

3.1.5 Auto-Scaling (HPA)

- **Target:** service-api Deployment
- **Min Replicas:** 4
- **Max Replicas:** 10
- **Metrics:**
 - CPU: 70% average utilization
 - Memory: 80% average utilization
- **Scale-down stabilization:** 300 seconds
- **Scale-up:** Immediate (0 seconds)

3.2 Data Flow

```
1. User Request → Internet
  ↓
2. NGINX Ingress (External IP)
  ↓
3. service-api ClusterIP Service
  ↓
4. Load balanced to one of 4 API pods
  ↓
5. API Pod processes request
  ↓
6. Database query via postgres-service DNS
  ↓
7. PostgreSQL Pod returns data
  ↓
8. API Pod formats JSON response
  ↓
9. Response returned to User
```

3.3 Self-Healing Mechanisms

Service API Tier

1. **Deployment Controller:** Maintains 4 replicas
 - Automatically replaces failed pods

Database Tier

1. **Statefulset Controller:** Maintains 1 replica
 - Automatically replaces failed pod
 - Preserves PVC for data persistence

4. Justification for Resources Utilized

4.1 Technology Stack Selection

Component	Selected	Justification
Language	Python 3.11	Widely adopted, excellent libraries, easy to read and maintain
Framework	Flask	Lightweight, flexible, minimal overhead for simple REST APIs
WSGI Server	Gunicorn	Production-grade, multi-worker support, efficient
Database	PostgreSQL 15	Enterprise-grade, ACID compliant, excellent Python support (psycopg2)
Container Base	python:3.11-slim	Small image size (~120MB), security patches
DB Connection	psycopg2 with pooling	Efficient connection reuse, reduced latency

4.2 Kubernetes Resource Selection

Resource	Type	Justification
API Workload	Deployment	Stateless, supports rolling updates, easy scaling
DB Workload	Deployment	Single replica with PVC for assignment/demo simplicity
API Service	ClusterIP	Internal service for Ingress, no direct external exposure
DB Service	ClusterIP	Internal only, database should never be exposed externally
External Access	Ingress (NGINX)	Simple, portable ingress for AKS and demo requirements
Storage	PVC (managed-csi)	Azure Disk CSI-backed persistent volumes on AKS

4.3 Resource Requests and Limits

Service API Pod

Resource	Request	Limit	Justification
CPU	100m	500m	Flask is lightweight; 100m minimum for steady state, 500m for request bursts
Memory	128Mi	256Mi	Python/Flask footprint ~100MB; buffer for request handling

PostgreSQL Pod

Resource	Request	Limit	Justification
CPU	100m	500m	Low query volume; 100m sufficient, 500m for complex operations

Resource	Request	Limit	Justification
Memory	256Mi	512Mi	PostgreSQL needs buffer cache; 256Mi minimum for stable operation

Storage

Resource	Size	Justification
PVC	1Gi	Sample data <1MB; 1Gi is sufficient for assignment

4.4 Replica Count Justification

Tier	Replicas	Justification
Service API	4	Assignment requirement; provides high availability and load distribution
PostgreSQL	1	Assignment requirement; single-master is simpler and sufficient for demo

4.5 HPA Configuration Justification

Setting	Value	Justification
Min Replicas	4	Assignment requirement
Max Replicas	10	Cost control; 150% headroom for traffic spikes
CPU Threshold	70%	Industry standard; leaves headroom for bursts
Memory Threshold	80%	Memory-bound before CPU typically
Scale-down window	300s	Prevents flapping during variable traffic

5. Deployment Instructions

5.1 Prerequisites

1. **Azure CLI** installed and authenticated
2. **kubectl** installed
3. **Docker** installed and authenticated to Docker Hub
4. **AKS Cluster** created

5.2 AKS Cluster Access (credentials)

```
# Get credentials for your AKS cluster
az aks get-credentials --resource-group <RESOURCE_GROUP> --name <AKS_CLUSTER_NAME>
```

5.3 Install NGINX Ingress Controller (use cloudshell inside AKS cluster)

```
helm repo add ingress-nginx https://kubernetes.github.io/ingress-nginx
helm repo update
helm install ingress-nginx ingress-nginx/ingress-nginx
```

Get the NGINX external IP:

```
kubectl get svc ingress-nginx-controller -w
```

5.3 Build and Push Docker Image

```
cd service-api

# Build image
docker build -t YOUR_DOCKERHUB_USERNAME/nagp-service-api:latest .

# Push to Docker Hub
docker push YOUR_DOCKERHUB_USERNAME/nagp-service-api:latest
```

5.4 Update Kubernetes Manifests

Edit `k8s/06-service-api-deployment.yaml`:

```
image: YOUR_DOCKERHUB_USERNAME/nagp-service-api:latest
```

5.5 Deploy to AKS

```
# Apply all manifests in order
kubectl apply -f k8s/

# Or use the deployment script
bash scripts/deploy.sh
```

5.6 Verify Deployment

```
# Check all resources
kubectl get all

# Get NGINX external IP (may take a few minutes)
kubectl get svc ingress-nginx-controller
```



```
# Test API
curl http://<NGINX_EXTERNAL_IP>/api/employees
```

6. Demonstration Guide

6.1 Show All Deployed Objects

```
# All resources
kubectl get all

# Persistent Volume Claims
kubectl get pvc

# Horizontal Pod Autoscaler
kubectl get hpa

# Ingress
kubectl get ingress
```

6.2 API Call - Retrieve Records

```
# Get NGINX external IP (AKS)
NGINX_EXTERNAL_IP=$(kubectl get svc ingress-nginx-controller -o
jsonpath='{.status.loadBalancer.ingress[0].ip}')

# Retrieve employees
curl http://${NGINX_EXTERNAL_IP}/api/employees | jq .
```

6.3 Self-Healing - API Pod

```
# List current pods
kubectl get pods -l app=service-api

# Delete one pod
kubectl delete pod <POD_NAME>

# Watch regeneration
kubectl get pods -l app=service-api -w
```

Expected behavior: Deployment controller immediately creates a new pod to maintain 4 replicas.

6.4 Self-Healing - Database Pod

```
# Delete database pod
kubectl delete pod -l app=postgres

# Watch regeneration
kubectl get pods -l app=postgres -w

# Verify data persistence after pod restart
curl http://${NGINX_EXTERNAL_IP}/api/employees
```

Expected behavior: Statefulset controller creates new pod, PVC preserves data, all 8 records intact.

6.5 Rolling Update Demonstration

```
# Update deployment image
kubectl set image deployment/service-api \
    service-api=YOUR_DOCKERHUB_USERNAME/nagp-service-api:v2 \

# Watch rollout
kubectl rollout status deployment/service-api

# During rollout, verify API remains accessible
curl http://${NGINX_EXTERNAL_IP}/api/employees
```

7. FinOps Analysis

7.1 Current Cost Estimate (AKS - high level)

Resource	Configuration	Monthly Cost (est.)
AKS Cluster	Node pool (size/count varies)	Varies by VM SKU + region
Managed Disk	1Gi Azure Disk	Low (region dependent)
Load Balancer	1 Service of type LoadBalancer (NGINX)	Varies (region dependent)
Network Egress	Based on traffic	Varies
Total		Depends on region/SKU

7.2 Three Cost Optimization Opportunities

Opportunity 1: Use Spot/Preemptible Nodes

Description: Use AKS Spot node pools to reduce compute cost during demos/non-critical workloads.

Implementation:

```
# Example (conceptual): create an AKS spot node pool
# az aks nodepool add --resource-group <RG> --cluster-name <AKS> --name spotpool -
-priority Spot --eviction-policy Delete --spot-max-price -1 --node-count 1 --node-
vm-size <VM_SIZE>
```

Savings: Up to 80% on compute costs

Trade-off: Pods may be preempted; mitigated by having 4 API replicas spread across nodes.

Opportunity 2: Right-size Resources Based on Metrics

Description: Monitor actual resource usage and adjust requests/limits.

Implementation:

```
# Check actual usage
kubectl top pods

# If API pods use only 50m CPU avg, reduce request to 50m and limit to 250m
```

Current vs. Optimized:

Resource	Current	Optimized	Savings
CPU Request	100m	50m	50%
Memory Request	128Mi	64Mi	50%

Impact: Lower requests allow more pods per node, potentially reducing node count.

Opportunity 3: Use AKS cluster autoscaler + right-size node pools

Description: Scale the node pool based on demand and pick VM SKUs that match observed usage.

Implementation:

- Enable AKS cluster autoscaler and tune node pool min/max
- Right-size the node VM size based on `kubectl top` + HPA behavior

Savings: Avoid paying for idle nodes while still supporting spikes.

7.3 Resource Optimization with Observed Metrics

To implement resource optimization based on observed metrics:

```
# 1. Deploy metrics-server (if not already installed)
kubectl apply -f https://github.com/kubernetes-sigs/metrics-
server/releases/latest/download/components.yaml
```

```
# 2. Collect metrics over time
kubectl top pods

# 3. View HPA metrics
kubectl get hpa -o wide

# 4. Analyze and adjust based on actual usage patterns
```

Recommended Monitoring Setup:

- Azure Monitor / Container Insights (AKS)
 - Custom Prometheus metrics for detailed analysis
 - Set up alerts for resource utilization thresholds
-

Summary

This solution fulfills all requirements of the NAGP 2026 Kubernetes assignment:

- Multi-tier architecture (Python Flask API + PostgreSQL)
 - Containerized with Docker, pushed to Docker Hub
 - Service API: 4 pods, rolling updates, external access, self-healing, HPA
 - Database: 1 pod, persistent storage, internal only, self-healing
 - Configuration via ConfigMap and Secrets
 - No Pod IPs used for inter-tier communication
 - External access via NGINX Ingress
 - Resource requests/limits defined (FinOps)
 - Three cost optimization opportunities identified
-

Document Version: 1.0